*Attach this Cover Page as the first page of your report PDF submission. Answer in your own words; keep each answer to the space provided.*

## A. Student and team particulars

| | |
|---|---|
| **Name / Student ID** | LE YI / G2605168D |
| **Undergrad University / Degree** | Zhengzhou University — B.Eng. in Industrial Design |
| **Partner's name and ID (if any)** | None (individual project) |
| **Tentative Plan after NTU Master** | To work in medical-device and healthcare-AI product roles. |

## B. Links (leave blank if not applicable)

| | |
|---|---|
| **Your CV Online and Github page** | https://github.com/Ean214 |
| **Online repository (e.g., Github code, data, tools)** | https://github.com/Ean214/MD6329 |
| **Demo or presentation video** | |
| **Overleaf / report link** | |
| **LecTrace script or trace link** | https://github.com/Ean214/MD6329/tree/main/lectrace |

| | |
|---|---|
| **Most relevant software tool in project** | PyTorch (from-scratch Transformer encoder); XGBoost for the tabular baseline; PyVene for the intervention probes |
| **Your project extends Assignments 1 or 2?** | Yes — Assignment 2. The Transformer encoder is adapted from my Assignment 2 implementation (sinusoidal positional encoding, key-padding masks, multi-head attention), and the interpretability probe follows its observation-versus-intervention lesson. |

| Describe the software bug in your deep learning program (e.g., Pytorch, Tensorflow or ConvNetJS Javascript etc) that cost you the most time. How did you find it, and how did you know it was fixed? Did AI coding agent help you to find bugs? |
|---|
| The most expensive bug was not a crash but a misleading measurement. I ran an ablation that zeroed a component of a clinical-sequence Transformer and found the output probability essentially unchanged (about 1e-6), which I first read as "this component does not matter". Most of a day went into that wrong conclusion. The metric was the problem: predictions had saturated near 0.9999, and there the sigmoid slope p(1-p) is about 5.7e-05, so any real change is compressed to nothing on the probability scale. I found it by re-running the same ablation on the logit scale and by manually bypassing the intervention library, to rule out a hook error. It was fixed once all ablation reporting moved to logit scale and I added a whole-test-set sanity check: zeroing the same component across the full test set moved accuracy from 1.0000 to 0.5000 and changed the mean logit by 9.65. An AI coding agent did help — it recognised the saturation pattern and suggested the logit-scale check — but it had initially agreed with my wrong reading, which is why the manual cross-check and the whole-set check mattered more than the explanation. In this project the same tool was useful again when a pandas group-by over 160,000 dialysis sessions repeatedly died; it proposed a vectorised rewrite that cut the runtime of four dataset builds to about nineteen minutes in total. |

| Which of the Lecture Videos (https://www.youtube.com/playlist?list=PLORG7zBQosxw) can be improved and why? |
|---|
| I would improve Lecture 6 (EHR Part 2: RNN/LSTM, around 02:14), where the vanishing-gradient problem and the gating mechanism are explained verbally but without a worked numerical example. For a student coming from a product and clinical background — my own case — the gate arithmetic only became concrete when I implemented one LSTM cell myself. A short animated walk-through of a single time step (candidate state, forget and input gates, cell-state update) at the point where it is introduced would remove the main stumbling block. A smaller secondary point: Lecture 7 covers attention thoroughly (around 00:54), but the distinction between attention weights as an explanation and as an intervening variable only appears much later; raising it once in Lecture 7 would prevent a very common misreading, and it is exactly the confusion I made myself. |

## C. LecTrace — execution-explanation presentation

| | |
|---|---|
| **Approximate time spent learning LecTrace** | About 2 hours — installation, one walk-through of the demo lecture, then writing and iterating on my own script. |

| | |
|---|---|
| **LecTrace script written by yourself or mostly by AI?** | Written by me; AI assistance was used for boilerplate and formatting. The content decisions — which execution states to show and why — are mine. Disclosed in the report's Software 3.0 section. |

**Q.** Which execution states did you choose to show, and why those rather than others?

I chose five states, and each one is a place where a silent mistake would invalidate a headline number.
1. Session reconstruction: one patient's raw readings being cut into sessions at a six-hour gap. The unit runs three shifts and the last one crosses midnight, so keying sessions on the calendar date splits them in two and quietly destroys the weight fields. Showing the cut makes the pitfall visible.
2. The leakage boundary at T = 60 minutes: the readings that go into the model, and the later readings that are used only to build the label. If this boundary slips, every score becomes meaningless.
3. The patient-level split: the same patient ID never appearing in two splits. Rows, unlike patients, are not independent.
4. One decision point from the control simulation: the candidate ultrafiltration values, the predicted risk for each, and the chosen candidate together with the extra fluid and time it costs. This is the paper's central trade-off in one screen.
5. The ablation that surprised me: zeroing attention versus zeroing the feed-forward block in the Transformer, and the resulting logit change (about 1e-08 versus 0.042). It is the reason I do not present attention maps as explanations.
I chose these rather than the loss curve because the loss curve is the least informative part of the work: it looks healthy even when the evaluation is wrong.

**Q.** What was confusing or missing in LecTrace, and what one feature would you improve or add?

My main friction was that stepping through a long computation is only useful if the interesting state is visible, and it took me a while to learn which variables to mark for inspection. The viewer shows what you ask for, so the pedagogical work happens before the trace, not inside it. A smaller issue is reproducibility: with a stochastic training loop, re-running a step yields different numbers unless the seed is fixed explicitly, so the trace is not replayable by default.
The one feature I would add is an assertion step: let a traced value be compared against an expected value and fail loudly when they differ. My five states exist precisely to catch silent errors — the label window, the patient split, the chosen candidate — so having the tool assert them would turn a presentation into a regression test. That would have caught my saturation bug in seconds rather than a day.

*By default, you consent to your anonymised answers being used in research on teaching. Let me know if otherwise. This choice doesn't affect your grade.*